바이브 코딩 7일 완성 — 클로드와 함께하는 첫 개발 · 7일차 부록

Git·GitHub 기초와 클로드 코드 연동 가이드

버전 관리의 개념부터, 클로드 코드에게 말로 커밋·푸시를 맡기는 법까지

대상 마케팅 실무자·마케터 (바이브 코딩 입문)
관련 일차 7일차 5교시(깃허브 업로드)·6교시(Vercel 배포) · 3일차 Git 설치 · 6일차 .gitignore
기준 환경 Windows 10/11 · Git for Windows · 클로드 코드 · GitHub 개인 계정

> 📌 **이 가이드가 필요한 순간**
>
> 7일차에는 GitHub MCP로 랜딩페이지 앱을 깃허브에 올리고, 그 저장소를 Vercel에 연결해 배포합니다. 수업에서는 클로드 코드가 대부분을 대신 처리하지만, **무엇이 어디에 저장되는지**를 알고 있어야 오류가 났을 때 당황하지 않고, 수업이 끝난 뒤에도 내 프로젝트를 스스로 관리할 수 있습니다. 명령어를 외울 필요는 없습니다. 개념을 이해하고 **클로드 코드에게 정확히 부탁하는 법**을 익히는 것이 목표입니다.

이 가이드에서 배우는 것

01 Git과 GitHub, 무엇이 다를까
02 꼭 알아야 할 용어
03 작업 흐름 한눈에 보기
04 처음 한 번만 하는 준비
05 클로드 코드에게 Git 작업 맡기기
06 GitHub와 연결하는 두 가지 방법
07 올리면 안 되는 것 — .gitignore와 보안
08 자주 겪는 문제와 해결
09 한 장 요약

> ✋ **핵심을 한 문장으로**
>
> **Git**은 내 컴퓨터에서 작업 내용을 시점별로 저장해 두는 **세이브 기능**이고, **GitHub**는 그 세이브 파일을 인터넷에 보관하고 공유하는 **클라우드 보관소**입니다.

01 Git과 GitHub, 무엇이 다를까

'기획안_최종.pptx', '기획안_최종_수정.pptx', '기획안_진짜최종.pptx' … 파일을 복사해 가며 버전을 관리해 본 경험이 있을 겁니다. **Git**은 이 일을 자동으로 해 주는 **버전 관리 도구**입니다. 파일은 하나만 두고, '언제, 무엇을, 왜 바꿨는지'를 기록으로 남겨 두었다가 원할 때 어느 시점으로든 되돌아갈 수 있게 해 줍니다.

GitHub(깃허브)는 Git으로 기록한 프로젝트를 인터넷에 올려 두는 서비스입니다. 컴퓨터가 고장 나도 코드가 안전하게 남고, 다른 사람과 공유하거나 Vercel 같은 배포 서비스와 연결할 수 있습니다.

구분	Git	GitHub
정체	내 컴퓨터에 설치하는 프로그램	인터넷 웹사이트(github.com)
하는 일	작업 내용을 시점별로 기록하고 되돌리기	기록을 온라인에 보관·공유·협업
비유	게임의 세이브 포인트	세이브 파일을 올려 두는 클라우드
인터넷	없어도 동작	필요
이 과정에서	3일차에 설치 (부록1 02장)	7일차에 가입하고 업로드

> 💡 **마케터에게 왜 필요할까요?**
>
> - **되돌리기 안전망** — 클로드 코드에게 큰 수정을 맡겼다가 결과가 마음에 들지 않으면, 커밋해 둔 시점으로 언제든 돌아갈 수 있습니다.
> - **배포의 출발점** — Vercel은 깃허브에 올린 코드를 가져가 자동으로 배포합니다. 코드를 다시 푸시하면 사이트도 자동으로 갱신됩니다.
> - **백업과 공유** — 노트북을 바꿔도 깃허브에서 그대로 내려받아 이어서 작업할 수 있고, 동료나 개발팀에 링크 하나로 전달할 수 있습니다.

02 꼭 알아야 할 용어

클로드 코드가 작업 중에 보여 주는 메시지에 자주 등장하는 단어들입니다. 처음부터 다 외울 필요는 없고, 볼 때마다 이 표로 돌아오면 됩니다.

용어	뜻	비유
저장소 (repository, repo)	Git이 기록을 관리하는 프로젝트 폴더. 폴더 안에 숨김 폴더 `.git`이 생기며 여기에 모든 기록이 담깁니다	기록 장치가 달린 폴더
로컬 / 원격 저장소	로컬은 내 컴퓨터의 저장소, 원격(remote)은 깃허브에 있는 저장소. 원격 저장소의 기본 이름은 보통 `origin`	내 PC 사본 / 클라우드 사본
추가 (add, 스테이징)	다음 커밋에 넣을 파일을 골라 준비 공간(staging area)에 담기	택배 상자에 물건 담기
커밋 (commit)	담아 둔 변경 내용을 설명 메시지와 함께 로컬 저장소에 확정 기록	상자를 봉하고 송장 붙이기
푸시 (push)	로컬에 쌓인 커밋을 깃허브(원격 저장소)로 업로드	택배 발송
풀 (pull)	깃허브에 있는 최신 커밋을 내 컴퓨터로 내려받아 합치기	택배 수령
클론 (clone)	깃허브 저장소를 기록까지 통째로 내 컴퓨터에 복제	클라우드에서 전체 내려받기
브랜치 (branch)	원본을 건드리지 않고 따로 작업하는 갈래. 기본 브랜치 이름은 보통 `main`	문서의 '사본으로 작업하기'

.gitignore	Git이 기록·업로드하지 않을 파일 목록. `.env` 같은 비밀 파일을 여기에 적습니다 (6일차)	'발송 금지' 목록
README.md	저장소 첫 화면에 표시되는 프로젝트 소개 문서	상품 상세 페이지

03 작업 흐름 한눈에 보기

7일차 교재의 '깃허브 작업 흐름 3단계(추가 → 커밋 → 푸시)'를 그림으로 보면 다음과 같습니다.

① 작업 폴더
클로드 코드가 파일을 만들고 고치는 곳 (예: `Study-07`)

→

② 로컬 저장소
`add`로 담고 `commit`으로 기록. 내 컴퓨터 안에 저장됨

→

③ 깃허브
`push`로 업로드. 인터넷에서 확인·공유·배포 가능

반대 방향으로, 깃허브의 내용을 내 컴퓨터로 가져올 때는 `pull`(이미 연결된 폴더) 또는 `clone`(처음 받을 때)을 씁니다.

> ✋ **여기서 잠깐 · 커밋만 하고 푸시를 안 하면?**
> 커밋은 **내 컴퓨터에만** 기록됩니다. 깃허브 웹사이트에 변경 내용이 보이지 않거나 Vercel 사이트가 바뀌지 않는다면, 대부분 푸시를 하지 않은 경우입니다. 클로드 코드에게 "커밋하고 **푸시까지** 해줘"라고 끝까지 요청하세요.

04 처음 한 번만 하는 준비

STEP 1 **Git 설치 확인**
새 터미널에서 `git --version`을 입력해 버전이 나오는지 확인합니다. 나오지 않으면 「3일차 부록1 · 클로드 코드 설치·실행 통합 가이드」02장을 따라 설치합니다.

STEP 2 **내 이름과 이메일 등록**
커밋 기록에 '누가 작업했는지' 남기기 위한 설정입니다. 컴퓨터마다 한 번만 하면 됩니다. 이메일은 깃허브 가입 이메일과 같게 맞추는 것을 권장합니다.

```
git config --global user.name "Hong Gildong"
git config --global user.email "gildong@example.com"
git config --global --list        :: 등록된 값 확인
```

클로드 코드에게 "내 Git 사용자 이름은 Hong Gildong, 이메일은 gildong@example.com으로 설정해줘"라고 부탁해도 됩니다.

STEP 3 **깃허브 가입**
https://github.com 접속 → [Sign up]으로 계정을 만듭니다. 7일차에는 노션·깃허브·Vercel·Supabase 가입이 모두 필요하니 **전날까지 미리** 가입해 두세요.

> ⛔ **회사·학교 계정은 피하세요**
> 회사나 학교의 SSO(통합 로그인)에 묶인 깃허브 계정은 토큰 발급이 조직 관리자 승인 대기로 막히는 경우가 많습니다. 실습은 **개인 이메일로 만든 개인 계정**으로 진행하세요.

05 클로드 코드에게 Git 작업 맡기기

Git이 설치되어 있으면 클로드 코드는 Git 명령을 직접 실행할 수 있습니다. 명령어를 몰라도 **하고 싶은 일을 말로 요청**하면 됩니다. 클로드 코드가 명령을 실행하기 전에 승인을 물으면 내용을 읽고 '1. Yes'를 선택합니다.

하고 싶은 일	클로드 코드에 이렇게 요청	실제로 실행되는 명령
이 폴더에서 버전 관리 시작	"이 폴더를 Git 저장소로 만들고, .env와 불필요한 파일이 올라가지 않게 .gitignore도 만들어줘"	`git init`
지금 상태 확인	"지금 Git 상태 알려줘. 커밋 안 된 변경 사항이 있어?"	`git status`
현재 작업 저장(세이브)	"지금까지 작업한 내용을 커밋해줘. 커밋 메시지는 한국어로 무엇을 바꿨는지 알 수 있게 써줘"	`git add` → `git commit`
지난 기록 보기	"최근 커밋 10개를 날짜와 메시지로 정리해서 보여줘"	`git log`
무엇이 바뀌었는지 보기	"마지막 커밋 이후에 뭐가 바뀌었는지 쉽게 설명해줘"	`git diff`
예전 상태로 되돌리기	"어제 '버튼 색 변경' 커밋 이후의 변경을 되돌려줘. 기록은 지우지 말고 되돌리는 커밋을 새로 만들어줘"	`git revert`
깃허브에 올리기	"변경 내용을 커밋하고 깃허브에 푸시해줘"	`git push`

> 💡 **좋은 습관 · 큰 작업 전에는 세이브부터**
> 디자인을 통째로 바꾸거나 기능을 크게 추가하기 전에 "지금 상태를 커밋해 둬"라고 먼저 요청하세요. 결과가 마음에 들지 않으면 "방금 커밋한 시점으로 되돌려줘"라고 하면 됩니다. 커밋은 자주, 작게 할수록 되돌리기 쉽습니다.

> 📘 **/rewind와 Git 되돌리기는 무엇이 다를까**
> - `/rewind` (또는 Esc 두 번) — 클로드 코드 대화 안에서 **클로드가 고친 내용**을 바로 이전 시점으로 되돌립니다. 지금 하던 대화 중의 실수를 빠르게 취소할 때 씁니다.
> - **Git 커밋** — 대화가 끝나도, 며칠이 지나도 남는 **영구 기록**입니다. 내가 직접 고친 내용까지 포함해 어느 커밋 시점으로든 돌아갈 수 있습니다.

> ⛔ **이런 승인 요청은 한 번 더 읽어 보세요**
> `git reset --hard`, `git push --force` (`-f`), `git clean`은 **기록이나 파일을 되살릴 수 없게 지울 수 있는** 명령입니다. 이런 명령의 승인을 묻는다면 바로 Yes를 누르지 말고 "이 명령을 실행하면 무엇이 사라지는지 설명해줘"라고 먼저 물어보세요. 되돌리기는 기록을 남기는 `git revert` 방식을 요청하는 것이 안전합니다.

06 GitHub와 연결하는 두 가지 방법

로컬 저장소의 커밋을 깃허브로 올리려면, 클로드 코드가 내 깃허브 계정에 접근할 수 있어야 합니다. 방법은 두 가지입니다.

구분	방법 A · GitHub MCP	방법 B · Git으로 직접 연결
사용처	7일차 수업에서 쓰는 방식	수업 이후 혼자 프로젝트를 관리할 때
인증	깃허브에서 발급한 토큰(Personal Access Token)	첫 푸시 때 브라우저로 깃허브 로그인
잘하는 일	저장소 만들기, 이슈·PR 관리 등 깃허브 웹 기능까지 클로드가 대신 처리	평소의 커밋·푸시·풀. 토큰 만료 걱정이 적음
주의할 점	토큰 유효 기간이 끝나면 다시 발급하고 재연결	저장소를 깃허브 웹에서 먼저 만들어야 함

방법 A — GitHub MCP로 연결하기 (7일차 방식)

자세한 화면 순서는 7일차 교재 5교시 'STEP 3. 깃허브(GitHub) MCP로 프로젝트 영구 보관하기'에 있습니다. 여기서는 흐름만 정리합니다.

STEP 1 **토큰 발급**

깃허브 프로필 → [Settings] → [Developer settings] → [Personal access tokens] → [Tokens (classic)] → [Generate new token (classic)]. 용도 이름(예: vibecoding-day8), 유효 기간 30 days, 권한 `repo` · `user` · `project`를 체크하고 발급합니다.

STEP 2 **토큰 보관**

발급 화면을 벗어나면 다시 볼 수 없으므로 바로 복사해 안전한 곳에 보관합니다.

STEP 3 **MCP 등록**

프로젝트 폴더의 터미널에서 아래 명령을 실행합니다. `<깃허브 API 토큰>` 자리에 발급받은 토큰을 붙여 넣습니다.

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ -H "Authorization: Beare
r <깃허브 API 토큰>"
```

STEP 4 **연결 확인과 업로드**

`claude`로 실행한 뒤 `/mcp`에서 github가 connected인지 확인하고, 아래처럼 요청합니다.

```
이 프로젝트를 위한 깃허브 저장소를 새로 만들고, 지금까지 작업한 파일을 모두 커밋해서
저장소에 푸시해 줘. 저장소 이름은 landing-lead-collector로 해 줘.
```

> ✋ **토큰이 만료됐다면**
>
> 30일이 지나 연결이 끊기면 깃허브에서 토큰을 새로 발급한 뒤, `claude mcp remove github`로 기존 등록을 지우고 STEP 3을 새 토큰으로 다시 실행합니다.

방법 B — Git으로 직접 연결하기 (수업 이후 추천)

Git for Windows에는 깃허브 로그인을 대신 처리해 주는 **Git Credential Manager**가 함께 설치됩니다. 그래서 토큰 없이 브라우저 로그인만으로 푸시할 수 있습니다.

STEP 1 **깃허브에서 빈 저장소 만들기**

깃허브 오른쪽 위 [+] → [New repository] → 저장소 이름 입력, 공개 범위(Public/Private) 선택 → README 등은 추가하지 않은 상태로 [Create repository]. 생성 화면에 보이는 주소(`https://github.com/내아이디/저장소이름.git`)를 복사합니다.

STEP 2 **클로드 코드에 연결 요청**

프로젝트 폴더에서 클로드 코드를 실행하고 요청합니다.

```
이 폴더를 https://github.com/내아이디/저장소이름.git 에 원격 저장소로 연결하고,
지금까지 작업을 커밋해서 main 브랜치로 푸시해줘.
```

STEP 3 **처음 한 번 로그인**

첫 푸시 때 'Connect to GitHub' 창이 뜨면 **[Sign in with your browser]**를 눌러 깃허브에 로그인하고 승인합니다. 이후에는 다시 묻지 않습니다.

> 📘 **Public과 Private, 무엇을 고를까**
>
> **Public**은 누구나 코드를 볼 수 있고, **Private**은 나와 내가 초대한 사람만 볼 수 있습니다. 고객 정보, 내부 기획 자료, 캠페인 전략이 담긴 프로젝트라면 **Private**을 고르세요. Private 저장소도 Vercel에 연결해 배포할 수 있습니다. 공개 범위는 나중에 저장소의 [Settings] 맨 아래에서 바꿀 수 있습니다.

07 올리면 안 되는 것 — .gitignore와 보안

한 번 깃허브에 올라간 내용은 나중에 파일을 지워도 **커밋 기록에 그대로 남습니다**. 그래서 올리기 전에 막는 것이 가장 중요합니다.

올리면 안 되는 것	이유와 처리
`.env`, API 키, 토큰	남이 내 계정으로 요금을 쓰거나 데이터에 접근할 수 있습니다. `.gitignore`에 반드시 등록 (「6일차 부록 · API 키 관리 가이드」)
고객 이메일·연락처가 담긴 파일	개인정보 유출이 됩니다. 실습 데이터도 가짜 값(test.com 등)만 사용
`node_modules`, `__pycache__` 같은 폴더	설치하면 다시 생기는 파일이라 올릴 필요가 없고 용량만 큽니다

푸시 전에 클로드 코드에게 이렇게 점검을 부탁하는 습관을 들이세요.

```
푸시하기 전에 .gitignore를 확인해서 .env, API 키, 개인정보가 담긴 파일이
커밋에 포함되지 않았는지 점검하고, 포함됐다면 제외한 뒤에 푸시해줘.
```

> ⛔ **이미 키를 올려 버렸다면**
>
> 파일을 지우고 다시 커밋하는 것만으로는 부족합니다. 기록에 남은 키는 이미 노출된 것으로 보고, **해당 서비스에서 키를 즉시 폐기하고 새로 발급**하세요. 그다음 새 키는 `.env`에만 넣고 `.gitignore`를 확인합니다.

> 📘 **CLAUDE.md에 규칙으로 적어 두기**
>
> CLAUDE.md의 '하지 말아야 할 것'에 ".env 파일을 git에 커밋하지 말 것", "푸시 전에 .gitignore를 확인할 것"을 적어 두면 클로드 코드가 매번 지켜 줍니다. (「3일차 부록3 · CLAUDE.md 작성법 가이드」 PART 5)

08 자주 겪는 문제와 해결

메시지 / 증상	원인과 해결
`git`은(는) 내부 또는 외부 명령이 아닙니다	Git이 설치되지 않았거나 설치 직후 창을 새로 열지 않았습니다. 터미널을 모두 닫고 새로 연 뒤 `git --version`으로 확인. 그래도 안 되면 Git 재설치
`Author identity unknown` `Please tell me who you are`	이름·이메일이 등록되지 않았습니다. 04장 STEP 2의 `git config`를 실행
`not a git repository`	아직 Git 저장소가 아닌 폴더입니다. 프로젝트 폴더가 맞는지 확인하고 "이 폴더를 Git 저장소로 만들어줘"라고 요청
푸시가 거부됨 `rejected … (fetch first)`	깃허브 쪽에 내 컴퓨터에 없는 커밋이 있습니다(웹에서 README를 만든 경우 등). "깃허브의 변경 사항을 먼저 pull 받아 합친 다음 다시 푸시해줘"라고 요청. 강제 푸시(`--force`)는 피하세요
`Authentication failed`, `403`	방법 A: 토큰이 만료됐거나 `repo` 권한이 빠졌습니다 → 토큰 재발급 후 재연결. 방법 B: 다른 깃허브 계정으로 로그인되어 있을 수 있습니다 → Windows [자격 증명 관리자]에서 github.com 항목 삭제 후 다시 로그인
`/mcp`에서 github가 failed / 연결 안 됨	토큰을 붙여 넣을 때 `<` `>` 기호까지 함께 넣었거나 앞뒤에 공백이 들어간 경우가 많습니다. `claude mcp remove github` 후 다시 등록
토큰 발급이 '승인 대기'로 막힘	회사·학교 SSO 계정입니다. 개인 계정을 새로 만들어 진행
`LF will be replaced by CRLF` 경고	Windows와 Mac의 줄바꿈 방식 차이를 알려 주는 안내일 뿐입니다. 무시해도 됩니다
깃허브에는 올라갔는데 Vercel 사이트가 그대로	Vercel 대시보드에서 배포가 진행 중인지 확인합니다. 시작 페이지 파일 이름이 `index.html`인지도 확인 (7일차 '404 오류' 안내 참고)

> 💡 **막히면 이렇게 물어보세요**
>
> 오류 메시지를 그대로 복사해 클로드 코드에 붙여 넣고 "이 Git 오류가 무슨 뜻인지 쉽게 설명하고, 기록을 지우지 않는 안전한 방법으로 해결해줘"라고 요청하세요.

09 한 장 요약

명령어	하는 일	클로드 코드에 이렇게 말하기
`git init`	버전 관리 시작	"이 폴더를 Git 저장소로 만들어줘"
`git status`	현재 상태 확인	"지금 Git 상태 알려줘"
`git add .` + `git commit -m "메시지"`	현재 작업 저장	"지금까지 작업 커밋해줘"
`git log --oneline`	기록 보기	"최근 커밋 목록 보여줘"
`git push`	깃허브로 업로드	"깃허브에 푸시해줘"
`git pull`	깃허브에서 최신 내용 받기	"깃허브 최신 내용 받아와줘"
`git clone 주소`	저장소 통째로 내려받기	"이 저장소를 내 컴퓨터로 받아줘: 주소"
`git revert`	기록을 남기며 되돌리기	"그 커밋을 되돌리는 커밋을 만들어줘"

✅ 7일차 전 체크리스트

- ☐ `git --version`으로 Git 설치를 확인했다
- ☐ `git config --global`로 이름과 이메일을 등록했다
- ☐ 개인 이메일로 깃허브 계정을 만들었다 (노션·Vercel·Supabase도 함께)
- ☐ `.env`가 `.gitignore`에 들어 있는지 확인하는 습관을 들였다

> 💡 **기억할 세 가지**
>
> ① **커밋은 세이브, 푸시는 업로드** — 커밋만 하면 깃허브에는 보이지 않는다. ② 큰 작업 전에는 **"지금 상태 커밋해 둬"**부터. ③ **키와 개인정보는 절대 올리지 않는다** — 푸시 전에 .gitignore 점검.

관련 자료: 「7일차 · MCP로 클로드 코드의 한계 넘어서기」 5교시 · 「3일차 부록1 · 클로드 코드 설치·실행 통합 가이드」(Git 설치) · 「3일차 부록3 · CLAUDE.md 작성법 가이드」 · 「6일차 부록 · API 키 관리 가이드」 · 「강사용 트러블슈팅 가이드」(7일차)
참고: Git 공식 문서(git-scm.com/doc), GitHub Docs(docs.github.com) · 2026년 10월 기준. 깃허브 메뉴 이름과 화면은 업데이트에 따라 조금 다를 수 있습니다.